Academic course project. Report version 8 November 2026. Not an IEEE publication.

Unpublished academic project report

Reinforcement Learning for Production Scheduling under Machine Wear and Electricity Demand Charges

ABHIMANYU T K, ADITH ANISH, VISHNU J, SWARAG PADIYANKUNNATH
[1]22AIE401 Reinforcement Learning course project. Abhimanyu T K AM.SC.U4AIE23102; Adith Anish AM.SC.U4AIE23103; Vishnu J AM.SC.U4AIE23160; Swarag Padiyankunnath AM.SC.U4AIE23058

All factory outcomes are simulated. This report uses the user-supplied IEEE Access template for academic formatting.

ABSTRACT We investigate factory scheduling when jobs have deadlines, machines deteriorate and electricity costs depend on both energy and billing-period demand peaks. A reproducible simulator combines benchmark-derived processing durations with synthetic routes, arrivals, failures and prices. A persistent partially observed formulation adds paid inspections and particle-based condition estimates. We compare masked neural proximal policy optimisation, readable cross-entropy policy search and strong scheduling rules. The original 60-day study contains 12,500 evaluated episodes and does not establish an adaptive-inspection advantage. A redesigned shared neural scorer lowers mean cost from 46.339 to 37.965 against the older neural control on a separate matched three-day test, an exploratory improvement of 18.1%. It remains weaker than a retuned rule. A grouped-action variant preserves physical trajectories but does not establish a cost gain. Final choices are locked on common validation before a fresh fixed-policy test; cross-entropy policy search has the lowest observed test mean, while the recorded overall validation winner remains the retuned rule. The contribution is an implemented and audited formulation, reproducible comparisons and a measured controller investigation, not a new proximal policy optimisation algorithm or proven industrial saving.

INDEX TERMS Action masking, condition estimation, demand charges, production scheduling, reinforcement learning.

I. INTRODUCTION

A factory must repeatedly choose which job to run on a free machine. A short operation may be attractive immediately, but switching product type adds setup time. Running a machine also changes its condition, failure exposure and electricity demand. A decision can therefore affect deadlines and costs long after the immediate operation finishes.

Our topic is sequential production scheduling under usage-related machine wear and electricity demand charges. We study reinforcement learning as a method for learning job decisions from simulated experience. We also evaluate simple rules carefully: a learning algorithm is useful only if its measured behaviour compares favourably with credible alternatives.

A. RESEARCH QUESTIONS AND CONTRIBUTIONS

The project contains a legacy single-shift environment, a persistent inspection research environment and later constrained neural-dispatch experiments. They share a factory story, but their observation schemas, control restrictions and evaluation horizons differ. This report identifies those differences rather than treating every result as a single model accuracy score.

The principal research question is when it is worth paying for condition information before producing or maintaining, given deadlines and electricity consequences across shifts. A second question asks whether a better neural representation and more useful training transitions improve dispatch. Neither question presupposes that RL must win.

Our implemented contributions are a reproducible cost-accounted simulator, public-information condition estimation, masked learning interfaces, controlled baseline comparisons, readable policy search, shared candidate scoring and a locked validation-to-test selection procedure. The strongest supported neural improvement is relative to our older neural control, not superiority to every tuned rule. [1]–[6], [16]

II. BACKGROUND

A job consists of ordered operations. An operation may have alternative eligible machines and different processing durations on those machines. The simulator selects a currently free machine; the controller chooses one eligible job from its dynamic shortlist. The same action index can refer to different jobs at different decision times.

Earliest due date (EDD) prioritises deadlines. Shortest processing time (SPT) prioritises short work. Our original SPT comparator minimises processing plus setup time and applies a belief-based maintenance rule. The stronger retuned comparator increases the weight on setup time and tunes the maintenance threshold using validation only.

Preventive maintenance restores physical condition but consumes productive time. Inspection instead provides a noisy measurement; it does not repair the machine. Corrective repair follows a failure and resumes interrupted work after repair. Treating inspection as maintenance would misrepresent both the transition model and its economic value.

An MDP assumes a state sufficient to determine the distribution of the next state and reward. Exact machine condition and deterioration rate are hidden here. The full inspection problem is therefore modelled as a POMDP, while the policy uses an approximate particle belief and compressed public observation. We do not prove that a finite vector of summaries is an exact belief-state MDP. [11], [12]

An energy charge integrates kWh at the tariff of each minute. A demand charge depends on the largest aligned 15-minute average power in the billing period. The charge must preserve its history: resetting the liability every day would change the scheduling problem and could give an incorrect learning reward.

III. RELATED WORK AND CURRENT TECHNOLOGIES

Learning to Dispatch uses graph representations and learned dispatch policies for job-shop scheduling. It motivates scoring eligible operations with a shared function, but our new network is a small shared MLP, not a reproduction of that graph architecture. Its performance is established by our own evaluation. [6]

PPO supplies the clipped policy-gradient update, while invalid-action masking restricts sampling and evaluation to admissible actions. Stable-Baselines3 and its contributed MaskablePPO provide the software implementation. These algorithms and libraries are existing technology, not project novelty. [2]–[5]

Particle filtering represents uncertainty through weighted samples. Our implementation applies this general idea to hidden health and wear-rate estimation, with survival evidence and noisy or censored measurements. The physical deterioration law, particle settings and tariffs are modelling choices in this project, not factory-calibrated measurements. [11]

Koutas, Bismut and Straub (2024) compare recurrent DRL and Monte Carlo tree search for inspection and maintenance under erroneous observations. Their belief-free recurrent method is different from our explicit particle-belief pipeline. This is related problem motivation, not a claim that we copied their architecture or reproduced their results. [12]

Two relevant 2026 developments are actor-critic pretraining and RL hyperheuristics. Kernbach et al. initialise the critic from returns of the pretrained actor; our controlled adaptation did not establish an isolated critic benefit. Lassoued et al. study state-dependent scheduling-rule switching and commitment. Our project instead includes a fixed public service gate and direct job scoring; it does not implement their complete hyperheuristic. [13], [14]

Residual RL combines a conventional controller with learned corrections. We adapt the concept to bounded job logits around a strong prior; the cited work concerns robotics and does not establish a scheduling gain here. Temporal abstraction supplies background for grouping forced actions. We verify our own cost and trajectory equivalence explicitly. [8]–[10]

Cross-entropy policy search is a population-based optimiser over policy parameters. Earlier RL applications include noisy CEM in Tetris. We use Gaussian sampling, elite selection, smoothed updates and a variance floor to optimise eight readable parameters. We do not claim to invent CEM or to reproduce the Tetris method exactly. [7]

IV. PROBLEM DEFINITION

The shop contains five machines by default and three product types. Each daily sequence releases 30 jobs; each job has one to three sequential operations. A product switch may require setup. Eligible-machine alternatives provide flexibility, and a fixed-size shortlist exposes up to ten ready jobs for the current machine.

The inspection configuration operates an eight-hour shift starting at 14:00. State persists across closures: unfinished jobs remain, deadlines age, and machine condition and billing history carry forward. Closed periods have zero operating energy and frozen physical wear in this simulation. These simplifications are part of the model.

The objective combines lateness, energy, incremental demand liability, downtime, inspection expense and unfinished work. Completion, on-time completion, breakdowns and belief coverage are reported beside cost. A policy that merely abandons work must not appear attractive because it avoids energy or service expense.

The longer primary experiment covers 60 simulated days with two 30-day billing periods. Three-day studies use an accelerated two-day billing period for faster experimentation. Their absolute costs cannot be compared directly to the 60-day costs. Synthetic INR-labelled prices are configurable parameters; the combined normalised objective is not a rupee saving.

V. MDP AND POMDP FORMULATION

A. Formal elements. We use a finite-horizon POMDP (X, A, T, Omega, O, R, gamma, H). The latent physical state X includes job progress and release status, machine modes and remaining work, true health and wear rate, calendar time, setup state, demand-window energy, billing peak and accumulated costs. Exogenous arrivals and shocks are sampled reproducibly. Unreleased job details and future disturbances are not ordinary policy input.

B. Observation. Public information includes arrived job candidates, durations, slack, setup, product and remaining operations; observable machine status; clock, tariff and demand history; and belief summaries for hidden condition. A particle filter approximates health and wear-rate uncertainty from operation history, survival and inspection measurements. The neural policy sees summaries rather than the entire posterior particle distribution.

C. Actions. For the inspection environment A has 13 positions: ten current candidate jobs, preventive maintenance, inspection and WAIT. Empty shortlist entries and inadmissible service decisions are masked. Masks depend on permitted public information. The environment selects the free machine; the policy assigns a job or service action to that selected machine.

D. Transitions. A selected job begins setup or processing. Minute integration accounts for energy, deterioration, stochastic failure and job progress. Preventive maintenance takes 30 minutes and restores health to 1. Repair takes 90 minutes and restores health to 0.8; interrupted work resumes. Inspection takes 5 minutes by default and adds noisy information plus configured direct expense. It does not restore health.

E. Condition update. Conceptually, b_next is proportional to measurement likelihood times the predicted belief under the transition model. Our weighted-particle approximation also conditions on observed survival. Empirical interval coverage evaluates approximation quality. A nominal 90% particle interval is not automatically a calibrated 90% guarantee.

The finite-horizon objective and base-environment reward are

$$\min_{\pi} \quad \mathbb{E}_{\pi}[C_H], \tag{1}$$

$$r_t = -(C_{t+1} - C_t), \qquad \gamma = 1, \tag{2}$$

$$\sum_{t=0}^{T-1} r_t = -C_H \quad (C_0 = 0). \tag{3}$$

Here H is the physical horizon and T is the realised number of decision transitions. The newer dispatch wrapper divides each reward by the episode length d in days, so its training return is $-C_H/d$. Evaluation costs remain unscaled. The cost is

$$C_H = L/480 + E/1000 + D/3000 + 0.2U/60 + I/1000 + 2N_H. \tag{4}$$

L denotes overdue-job minutes; E energy expense; D accrued demand liability; U service downtime in minutes; I direct inspection expense; and N_H unfinished released jobs. A billing reset must not decrease accrued D.

G. Variable decision duration. A transition may advance several physical minutes. In the grouped variant, one outer step also executes consecutive single-action gated decisions and sums their rewards. Gamma 1 preserves the realised undiscounted objective; GAE lambda 0.98 now decays per grouped outer step, so the learning estimator changes. This is a temporal-representation experiment, not an assertion of identical optimisation updates. [9], [10]

H. State schemas. The legacy configuration supplies 233 values and 11 actions without explicit waiting. The full inspection configuration supplies 307 values and 13 actions. The newer constrained dispatch wrapper supplies 92 bounded values while retaining 13 positions under a tighter public service mask. Checkpoints from these schemas are not interchangeable.

VI. PROPOSED NEURAL METHODOLOGY

A. Legacy and inspection PPO. The original inspection learner uses an MLP with two 64-unit hidden layers. A public-rule imitation warm start precedes MaskablePPO. A separate imitation-only control measures whether subsequent PPO adds value. Validation may select the initial checkpoint; selected_steps=0 is disclosed as retained imitation, not a PPO improvement.

The original teacher uses EDD job ordering and an uncertainty/service rule. High overall imitation agreement does not imply good maintenance recall or good scheduling. Earlier diagnosis found that rare maintenance actions were poorly imitated and the critic initially remained random. Those observations motivated controlled changes; they do not identify a single cause of failure.

B. Shared neural scorer. Six candidate features describe duration, setup, slack, age, remaining operations and nominal energy. A shared two-layer 64-unit network scores each candidate using its features and 12 public context values. Sharing weights avoids separate meanings for fixed shortlist positions. A separate pooled critic estimates value. Job and context inputs are bounded before learning.

The 92-value wrapper contains ten blocks of eight values (six candidate features, prior logit and presence), plus 12 context values. In the no-prior arm, job-priority logits are ignored. Its public rule still controls maintenance and WAIT. Inspection and discretionary waiting while work is ready are suppressed by this gate; this experiment is not a learned joint service controller.

C. Residual arm. Job logits combine a frozen, previously validation-retuned SPT prior and a bounded neural residual. The residual output head starts at zero, making the initial deterministic policy exactly reproduce the rule. Stochastic training still explores other jobs. The residual can reverse nearby decisions, but its bound limits departures across large prior-score gaps. A matched no-prior arm tests the importance of that prior.

A. Actor and critic roles. The actor maps the current public observation to scores for legal actions. A masked distribution gives inadmissible actions no sampling probability. During training the actor samples choices, allowing experience with alternatives. During deterministic evaluation it chooses the highest-scoring legal choice. The critic separately estimates expected future return, using the training reward scale (negative accumulated cost per day in the newer dispatch wrapper). The critic helps reduce noisy learning updates; it does not supply a labelled optimal schedule. [2]–[5], [10]

An observation, action, reward, old action probability and value prediction are stored in a rollout. Generalized advantage estimation combines temporal prediction errors to estimate whether an action led to better return than the critic expected. Positive advantage encourages the sampled action and negative advantage discourages it, subject to the PPO objective. Here discount one preserves the finite-horizon cost objective. GAE lambda controls how successive errors are combined; it is an estimator parameter, not an extra reward component.

A. PPO UPDATE AND ALGORITHM

With advantage estimate $\hat{A}_t$ and probability ratio

$$\rho_t(\theta) = \pi_\theta(a_t \mid o_t)/\pi_{\theta_{old}}(a_t \mid o_t), \tag{5}$$

the clipped surrogate is

$$L^{clip}(\theta) = \mathbb{E}_t\big[\min(\rho_t \hat{A}_t, \operatorname{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)\big]. \tag{6}$$

The critic is trained separately by value loss. Clipping discourages large probability changes; it does not guarantee better scheduling. [2], [10] **Algorithm 1. Masked dispatch training and selection.**

1: Initialise shared actor and pooled critic; include step zero.
2: **for** each training rollout **do**
3: Generate training-only scenarios and public observations.
4: Apply public service gate and admissible-action mask.
5: Sample an action and integrate the physical transition.
6: Store reward, old probability and value prediction.
7: Estimate advantages; update actor and critic with PPO.
8: Evaluate checkpoints on separate validation scenarios.
9: **end for**
10: Lock the validation-selected checkpoint before fresh testing.

VII. READABLE POLICY SEARCH AND SYSTEM DESIGN

A. A policy can learn without a neural network. CEM searches a small vector of policy parameters using whole-episode outcomes. The readable policy combines six normalised job features in a weighted score and selects the eligible job with the smallest score. Two additional thresholds control service. The resulting eight-dimensional parameter vector is stored directly as numbers. This is Gaussian cross-entropy episodic policy search, an established optimisation idea adapted to the factory model; it is not PPO, a new algorithm or supervised classification. [7], [16]

The six score terms are processing duration divided by sixty, setup divided by thirty, slack divided by shift length and clipped to the configured range, clipped job age in days, remaining operations divided by the maximum, and a nominal energy-related term based on duration, power and current tariff. Normalisation matters: coefficient sizes cannot be interpreted independently of these feature scales. Correlated features also make a large coefficient insufficient evidence that one physical factor causally improves performance.

B. Exact service semantics. The seventh parameter is a mean-health maintenance threshold. The eighth is a posterior-standard-deviation inspection threshold. When inspection is legal and estimated condition spread exceeds its threshold, this policy inspects before checking the maintenance rule. When maintenance is legal and estimated mean condition falls below its threshold, it maintains. Otherwise it dispatches a legal job using the weighted score, or waits when no job can be assigned. These thresholds are readable heuristics, not a computed optimal value of information.

C. Population updates. At each search iteration, candidate vectors are drawn from a Gaussian distribution and clipped to declared parameter bounds. The incumbent, initial policy and current mean remain candidate anchors. All candidates share the sampled training scenarios for that iteration, making their fitness comparison less sensitive to different random workloads. Fitness is average complete-episode objective. The lowest-cost quarter forms the elite set. The mean and standard deviation move toward elite statistics through smoothed updates, with a nonzero variance floor preserving exploration.

A training-selected nominee is then evaluated on separate validation scenarios. The saved policy changes only when validation improves; the initial rule remains eligible. This separation matters because selecting the best-looking test parameter vector would contaminate the performance estimate. The search originally generated three saved policies from different optimiser seeds. A later common-validation comparison selected seed one before the fresh final test. That final test therefore compares fixed selected policies, not a fresh search on its test scenarios.

The system has four stages: reproducible scenario generation; physical simulation and public condition inference; rule, neural or CEM control; and independent evaluation. The environment integrates physical processes between decisions and emits observations, legal masks and realised costs. The controller cannot read true health or unreleased routes through the ordinary policy interface.

A scenario seed fixes the generated jobs and physical disturbance streams. Training seeds control neural initialisation and training scenario order. Separate filter and sensor streams support reproducibility. Different action sequences use matched physical shock tables, allowing paired scenario comparisons without exposing future shocks to a controller.

The experiment layer stores raw episodes, daily ledgers, billing ledgers, validation histories, selected checkpoints and checksums. An OS lock prevents duplicate coordinators. Source/configuration and checkpoint identities prevent incompatible cache reuse. Changed protocols have separate named output stages rather than overwriting earlier evidence.

Figure 1 in the design appendix shows the POMDP information boundary and the action–simulation–reward loop. A second diagram shows the later hybrid controller. Its service gate is a restriction of control; it is not equivalent to learning all original inspection actions.

For eligible job j, the readable score is $s_j = w^\top f_j$ and the policy selects $\arg\min_j s_j$. Six feature weights and two service thresholds are searched. Inspection checks posterior spread; maintenance checks mean condition. These are heuristics, not optimal information-value calculations. **Algorithm 2. Gaussian cross-entropy policy search.**

1: Initialise parameter mean, spread and incumbent rule.
2: **for** each population iteration **do**
3: Sample bounded vectors and shared training scenarios.
4: Evaluate full-episode costs; select the lowest-cost quarter.
5: Smooth mean and spread toward elite statistics.
6: Retain a nonzero variance floor; validate one nominee.
7: Save an improvement only when validation cost decreases.
8: **end for**
9: Freeze saved weights before independent test evaluation.

VIII. DATASET AND EXPERIMENTAL SETUP

A. Dataset origin. Taillard benchmark generators and published seeds provide processing-duration matrices. We use train families ta01–ta06, validation ta07–ta08 and test ta09–ta10. The generator was checked against the published 4 by 4 verification matrix. The benchmark is well established and has been used before; the project does not introduce Taillard as a new dataset. [1]

Our scenarios are benchmark-derived extensions, not unmodified Taillard instances. Routes, alternative machines, arrivals, deadlines, products, speeds, condition, failures and prices are synthetic or configured. Consequently, a result here must not be compared with published Taillard optimal makespans. There are no real factory sensor labels or industrial calibration measurements.

B. Input records. A job-input record describes a job, arrival, due time, product, operation index and an eligible machine-duration alternative. Multiple rows may belong to one operation. Dynamic health beliefs, actions, rewards and completed schedules are generated by interaction and belong to traces or episode metrics, not to a static labelled training table.

C. Split discipline. Training samples scenarios 0–999. The original validation range is 10000–10499 and the original test range 20000–20499. Additional named studies use distinct specified validation/test sequences. A scenario seed fixes one synthetic factory sequence; a training seed is a separate source of model variation. A policy–seed–scenario row is not an independent real-factory observation.

D. Episode counts. A three-day sequence has 30 jobs per day, or 90 released jobs. A 60-day sequence has 1800. The 12500-row study does not mean 12500 distinct factory datasets: rule rows span 500 scenarios, and neural rows additionally repeat scenarios across five training seeds and several information conditions.

E. Modules. inspection.py implements physical transitions, masks, observations and ledgers; belief.py implements inference; inspection_study.py implements the original learning and paired evaluation; priority_search.py implements CEM; refinement.py tests critic pretraining; residual_dispatch.py implements shared scoring and constrained service; event_dispatch.py groups forced actions; instances.py implements benchmark-derived generation. [16]

F. Runtime. Python, NumPy, Gymnasium, PyTorch, Stable-Baselines3 Contrib, pandas and Matplotlib are the principal technologies. The older CPU environment is preserved. A separate runtime uses PyTorch 2.11.0 with CUDA 12.8 on an RTX 4050 Laptop GPU with 6 GB memory. The simulator and tiny inference calls remain on CPU; batch gradient updates run on GPU. [4], [5], [15]

The actual 2048-step PPO benchmark measured approximately 631 decisions/s on CPU, 410 with full GPU execution and 579 with CPU collection and GPU updates. Mixed execution was selected for actual GPU learning with limited overhead, not because it beat CPU throughput. Hardware acceleration does not itself establish a better learned policy.

The original full-cycle protocol trained 15 neural models: adaptive, no-inspection and exact-health conditions across five seeds, each for 65536 PPO steps. Five rules and four neural policy labels (including imitation-only) give exactly 12500 evaluated rows on 500 common 60-day scenarios. All selected checkpoints and source/model identities were checked.

The shared-scorer study trained residual and no-prior arms across three seeds at 65536 transitions per arm/seed. Validation used 10200–10209; test used 24500–24549. Old adaptive and CEM controllers and two rules were evaluated on the same scenarios, giving 700 rows. Additional standalone-neural comparisons were computed after the primary study and labelled exploratory with unadjusted intervals.

TABLE 1. Declared shared-scorer training settings; no new training was launched for this report.

Setting	Shared scorer
Learning rate	0.0001
Discount / GAE lambda	1 / 0.98
Epochs / batch size	4 / 256
Clip / KL target	0.1 / 0.015
Rollout	4 environments × 512 steps
Budget	65536 transitions / seed

IX. PERFORMANCE EVALUATION METRICS

The user-supplied performance guide asks for episodic return, success, regret, convergence, sample efficiency, generalisation, seed stability and computational efficiency, together with benchmarks, ablations and robustness. We distinguish endpoints reconstructable from existing artifacts from analyses that were not established by the saved protocols. Illustrative curves and EV or gaming examples in the guide are not our results. [17]

A. Mean episodic return. Return sums episode rewards. The base environment uses negative incremental cost; the newer dispatch wrapper also divides by episode days. With discount one, its return is $-C_H/d$, whereas base-equivalent return is $-C_H$. The checkpoint curve uses the three-day training scale. The table labels negative full cost as base-equivalent return. Both are derived from costs, not separate reward logs. Compare only within a declared protocol.

B. Job performance and episode success. Job completion is completed released jobs divided by released jobs. On-time rate counts released jobs completed by their deadlines. These are distinct from an episode-level binary success. For an additional descriptive analysis we define an all-complete episode as completed_jobs equal to released_jobs with zero invalid actions. This definition was added after the experiments; it is not a predeclared optimisation endpoint or evidence that deadlines were all met.

C. Reference cost gaps and regret. We report paired excess cost relative to SPT, the tuned rule and learned controls. An optimal policy is unknown, so these differences are not optimal cumulative regret. Summing gaps over an arbitrary ordering of independent test scenarios would not demonstrate an online learning-regret bound. No such bound or optimal-oracle result is claimed.

D. Learning progress and sample efficiency. Checkpoint curves derive mean validation return as negative cost divided by three against environment transitions. A convergence threshold and persistence criterion were not declared before training, so selected steps are checkpoint choices rather than convergence speeds. Different controls use different teachers, representations or service gates. No controlled claim that one algorithm reaches an equal-quality target with fewer samples is established.

E. Generalisation, stability and compute. Seed stability is assessed using three separately trained scorers on common scenarios. Common-validation-to-fresh-test cost shifts are descriptive and conditional on selected policies; a matched train-versus-test generalisation gap was not measured. The zero-shot long-horizon transfer is separate exploratory evidence. Measured decisions per second and actual CUDA update counts describe computation; energy use, inference-only latency and peak GPU memory were not benchmarked.

TABLE 2. Metric availability and claim boundaries. Unavailable analyses are not replaced with illustrative numbers.

Metric	Reported evidence
Return / task cost	Declared scaling; raw objective
Episode success	Post-hoc all-complete counts
Seed variation	Three training seeds
Regret / convergence	Not established
Sample efficiency	No equal-target claim
Generalisation	Fresh tests; limited transfer

X. EVALUATION PROTOCOL AND LEARNING BEHAVIOUR

The grouped study used a 65536 physical-action target per seed/condition rather than giving it 65536 grouped choices. Validation used 10220–10229; test used 24700–24749. Its saved plain counterparts and two rules give another 700 rows. Completed-rollout overshoot and the final discarded collection probe are recorded separately.

Final selection compared six standalone neural checkpoints, three CEM policies and two rules on common validation 10240–10249: 110 validation records. Neural, CEM and overall choices were locked before test 24900–24949. Four fixed controllers then produced 200 fresh test rows. A lower test mean cannot retrospectively change the locked overall validation winner.

For multi-seed comparisons, the paired bootstrap resamples training seeds and common scenario sequences with 2000 draws and fixed bootstrap RNG seed 702. For final fixed selected policies, only scenario variation is represented. These latter intervals do not include training-seed or model-selection uncertainty. All results remain simulation evidence, with limited seed counts and no industrial external validation.

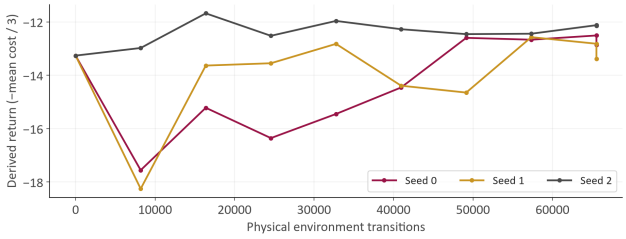


FIGURE 1. Actual standalone checkpoint means. Return is derived as negative validation cost divided by three. Curves are not monotonic; no predefined convergence threshold is available.

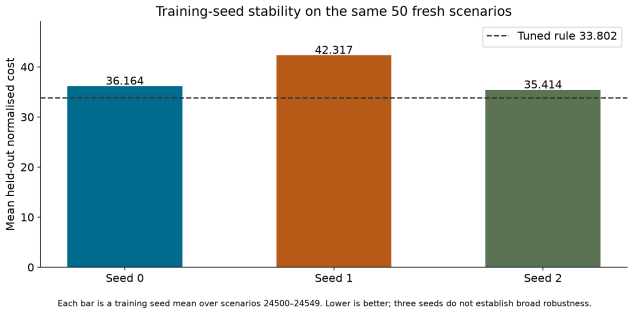


FIGURE 2. Each bar averages 50 common held-out scenarios for one training seed. The dashed line is the stronger tuned comparator.

The three shared-scorer seed means are 36.164, 42.317, 35.414; their sample standard deviation is 3.788. This is variation among three seed means, not an episode confidence interval. The same scenarios are evaluated for every seed, and three initialisations do not establish broad training robustness.

Validation selects steps 65536, 57344 and 16384 for no-prior seeds zero, one and two. All residual arms retain their initial prior. Choosing the best-looking test seed would overstate performance; the later final selection instead uses common validation. The substantial checkpoint fluctuations also make an ideal smooth convergence illustration inappropriate.

XI. RESULTS FROM THE ORIGINAL PERSISTENT STUDY

A. Original persistent study. Adaptive neural cost is 965.963 versus 652.847 for SPT with belief maintenance. Adaptive minus SPT is +313.117 with a supplemental paired 95% interval [277.388,355.844]. Adaptive is also worse than the no-inspection neural condition at 846.824. Four adaptive seeds retained imitation; across all information conditions 11 of 15 selected models retained step zero. No reliable added PPO benefit or inspection advantage is established.

The adaptive controller completes 99.807% of jobs, but only 44.37% on time. SPT completes 99.929%, with 60.67% on time. High completion is therefore not equivalent to good deadline performance. Approximate nominal 90% condition-interval coverage is 87.19% for adaptive and 89.88% for SPT; undercoverage is an inference limitation rather than AI accuracy.

TABLE 3. Original 60-day study. Five training seeds, 500 common scenarios and 12500 episode rows.

Controller	Cost	Complete	On time
Adaptive neural	965.963	99.81%	44.37%
Imitation only	991.631	99.83%	42.68%
No-inspection neural	846.824	99.86%	46.68%
SPT + belief PM	652.847	99.93%	60.67%

C. Inspection does not repair. A default inspection consumes five minutes and a direct configured expense of one hundred synthetic INR-labelled units. It updates information but does not restore health. Preventive maintenance instead consumes thirty minutes and restores health to one. Corrective repair after failure consumes ninety minutes and restores health to 0.8, after which interrupted work resumes. These values are simulator assumptions. Real maintenance durations, sensor accuracy and degradation would need calibration before industrial application.

Paid information is useful only if the improved belief changes subsequent decisions enough to offset the inspection time and expense. More accurate estimates can still produce a worse schedule if they arrive too late, cause unnecessary service, or distract from setup and deadline priorities. Accordingly, the reward supplies no inspection bonus. Policies are evaluated on the same realised objective, and inspection counts are reported so a cost comparison cannot silently be presented as evidence of information use.

D. Coverage and observed findings. A nominal ninety-percent condition interval should be assessed by the frequency with which simulated true health falls inside it. The original adaptive condition achieved approximately 87.19 percent coverage, while SPT achieved approximately 89.88 percent. These figures describe empirical inference coverage, not scheduling accuracy or calibrated certainty for every state. Undercoverage may reflect particle approximation, a changing action distribution or model assumptions and must remain a limitation.

The original sixty-day adaptive controller cost 965.963, compared with 652.847 for SPT with belief-based maintenance. It also cost more than the no-inspection neural control. The exploratory inspection grid and its fresh-scenario confirmation did not establish an inspection advantage. Later CEM policies and the constrained neural-dispatch studies used zero inspections. Their stronger scheduling results therefore cannot be used to rescue the original information-value hypothesis. A future test should predeclare sensor, cost and workload conditions and compare separately optimised controls without using final test outcomes to choose the protocol.

Checkpoint selection explains an important limit. Adaptive seeds zero, one, two and four retained their imitation warm start, while seed three selected 8192 PPO steps. Across all original information conditions, eleven of fifteen selected models retained step zero. The evaluated policy is therefore a validation-selected training pipeline; calling every retained policy a successfully improved PPO learner would be inaccurate.

The original adaptive minus imitation cost difference is approximately negative 25.667, with paired interval negative 78.741 to zero. Adaptive minus no-inspection cost is positive 119.139, with interval 86.163 to 155.472. These comparisons do not demonstrate an added PPO benefit or valuable inspection. The exact-health condition has privileged information but uses a particular learned controller and is not an optimal upper bound.

The earlier final pilot remains a separate negative result: adaptive cost 47.683 versus SPT 38.125, with four adaptive seeds retaining pre-PPO checkpoints. A ninety-six-cell exploratory inspection search and fresh-scenario confirmation also failed to establish an inspection advantage. These studies are retained to prevent a later favourable dispatch comparison from being mistaken for a successful original information-value experiment.

XII. SHARED SCORING AND GROUPED-ACTION RESULTS

B. Controlled neural improvement. On 50 fresh three-day sequences, the new no-prior scorer averages 37.965 versus 46.339 for the older adaptive neural controller, about 18.1% lower. The exploratory paired difference is -8.374 with 95% interval [-12.978,-3.363]. Completion rises from 97.104% to 98.511%, and on-time completion from 50.34% to 55.92%. The whole protocol changes; the gain cannot be attributed solely to network sharing.

Against retuned SPT, the new standalone scorer remains worse: difference +4.163 [1.032,7.973]. Its difference from original SPT is +1.149 [-2.164,5.065], which does not establish superiority. All residual arms retain step zero and exactly reproduce the strong rule. Their 33.802 cost is not an added PPO gain. Standalone seed costs vary substantially, so reporting only the best test seed would overstate robustness.

TABLE 4. Matched shared-scorer experiment on 50 fresh three-day scenarios.

Controller	Cost	Complete	On time
Older neural	46.339	97.10%	50.34%
Shared neural PPO	37.965	98.51%	55.92%
Retuned SPT	33.802	99.07%	60.53%
Learned CEM	33.850	99.06%	62.02%

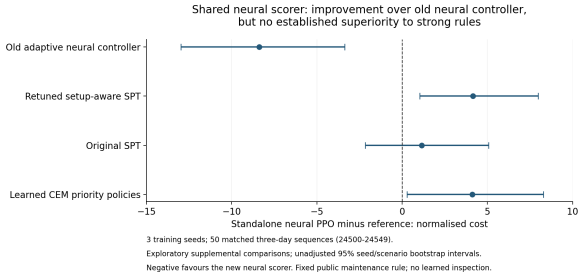


FIGURE 3. Exploratory paired cost differences, with seed and scenario uncertainty. Negative favours the standalone neural scorer.

C. Grouping forced decisions. Training-only diagnostics found 66.41% of sampled transitions allowed only one action. Ten paired trajectories verified identical physical actions, states, traces, ledgers and summed rewards after grouping. In the full study, grouped no-prior mean cost is 40.446 versus 40.063 for plain no-prior. Difference +0.383 [-2.510,3.629] does not establish a cost improvement.

Grouped on-time completion is 59.72% versus 55.19%, but completion is 98.119% versus 98.489%. These descriptive tradeoffs do not reverse the combined-cost finding. Completed training rollouts contain 65638–66989 physical actions, with 102–1453 overshoot and 4–65 extra discarded probe actions. Outer steps and physical steps must not be confused. All grouped residual arms again retain their prior.

TABLE 5. Grouping preserves realised physics, but does not establish a cost improvement.

Controller	Cost	Complete	On time
Grouped neural	40.446	98.12%	59.72%
Plain neural	40.063	98.49%	55.19%
Retuned SPT	37.106	98.84%	58.96%

D. Checkpoint selection is part of the method. Validation evaluates saved checkpoints, including step zero. In the original study step zero means the imitation warm start, not a learned PPO improvement. In the residual dispatch study it means the initial rule-assisted scorer. All three residual seeds retained step zero. The no-prior seeds selected steps 65536, 57344 and 16384, demonstrating that their selected policies actually included PPO training. Their achieved costs still require comparison with strong controls.

XIII. FINAL SELECTION SUCCESS AND FRESH-TEST BEHAVIOUR

D. Locked final check. Common validation selects plain no-prior seed 0 at 65536 steps as the neural model, CEM seed 1 as the readable learned policy and retuned setup-aware SPT as the overall validation winner. On the fresh fixed-policy test, CEM has the lowest observed mean, 34.669; the selected neural policy has 39.733. The recorded overall validation choice remains unchanged.

Selected neural minus retuned SPT is +3.425 [1.875,5.357]; neural minus CEM is +5.064 [2.769,7.434]. Neural minus original SPT is +1.961 [-0.497,4.584]. These are fixed-policy scenario intervals, not a multi-seed algorithm comparison. This test does not establish neural superiority. The earlier 18.1% gain belongs to its own matched experiment and cannot be transferred to this separate test.

TABLE 6. Four fixed policies on 50 fresh test scenarios. These are not a multi-seed algorithm comparison.

Controller	Cost	Complete	On time
Selected neural	39.733	98.42%	49.20%
Selected CEM	34.669	98.87%	61.84%
Retuned SPT	36.308	98.87%	60.13%
Original SPT	37.772	98.51%	60.38%

TABLE 7. Post-hoc episode success and base-equivalent return (negative full cost). All-complete does not imply all jobs were on time.

Policy	All-complete episodes	Base-equivalent return
Selected neural	26 / 50 (52%)	-39.733
Selected CEM	27 / 50 (54%)	-34.669
Tuned SPT	26 / 50 (52%)	-36.308
Original SPT	24 / 50 (48%)	-37.772

TABLE 8. Descriptive validation-to-test shift; not a measured training-to-test generalisation gap.

Policy	Validation cost	Fresh-test cost	Shift
Selected neural	36.973	39.733	+2.760
Selected CEM	35.284	34.669	-0.615
Tuned SPT	35.264	36.308	+1.045
Original SPT	36.279	37.772	+1.493

The binary success definition is reconstructed from completed and released job counts, with zero invalid actions. It is retrospective and should be interpreted beside job completion and on-time performance, not promoted to a new primary endpoint. These counts describe the frozen selected policies on fifty scenarios. Small samples and model-selection uncertainty limit broader conclusions.

The common-validation and test sets have different scenarios and benchmark families. A negative shift can occur through sampling and selection as well as actual transfer behaviour. The CEM test mean does not retrospectively replace the tuned rule as overall validation winner. Training-like return was not recorded under the same locked evaluation procedure, so this table is explicitly a validation-to-test shift rather than the guide's train-versus-unseen generalisation gap.

XIV. ABLATION ROBUSTNESS AND SENSITIVITY ANALYSIS

A. Component interventions. An ablation is useful only when the changed component and the comparison protocol are explicit. The original inspection/no-inspection comparison, the residual/no-prior comparison, the critic refinement and forced-action grouping ask different questions. Their results cannot be pooled as interchangeable trials of one complete method. [16], [17]

The readable-policy audit evaluates saved coefficients after removing energy, urgency/age or learned service parameters. These are fixed-policy interventions without retraining, so they test dependence of those saved policies rather than the best achievable policy after removing a feature. All learned readable policies used zero inspections. Disabling inspection therefore produces identical schedules here and does not establish that inspections are universally useless.

The audit costs are shown below. Removing the energy term does not worsen the mean, and changing service to the declared fixed rule also lowers it. The paired audit does not establish isolated benefits from energy, urgency/age or learned service thresholds. A claim that every added feature contributes positively is unsupported.

TABLE 9. Supplementary interventions on the same original priority-search scenarios. Means alone do not establish causal feature benefits.

Readable-policy variant	Mean cost
Saved learned priority	35.656
No inspection	35.656
Without energy	35.087
Without urgency / age	35.592
Fixed service	34.794
Validation-retuned SPT	35.995

In critic refinement, actor-critic pretraining cost is 44.263 versus 44.042 for actor-only and 36.904 for SPT. All six revised learners selected nonzero PPO steps, but the critic-specific interval crosses zero. In the shared-scorer residual arm, all selected steps are zero, giving exactly the strong prior; its favourable score cannot be attributed to an added neural improvement. The grouped-action cost interval likewise crosses zero.

B. Limited long-horizon transfer. Three saved readable policies were evaluated without retraining on ten new sixty-day sequences with thirty-day billing. Their mean combined cost is 592.431 versus 665.974 for original SPT, approximately eleven percent lower, with paired difference interval negative 100.594 to negative 46.554. The tuned rule averages 597.897; the learned-minus-tuned interval negative 25.904 to 20.637 crosses zero. These are exploratory simulation results from a small transfer set.

This transfer comparison is not the five-hundred-scenario original full-cycle result and does not validate the final selected neural checkpoint on sixty-day workloads. No broad matrix of workload, tariff, failure and sensor-noise distribution shifts has been completed for that final checkpoint. The inspection exploration is parameter sensitivity of a separate protocol, not such a neural robustness test. A future study should lock a shift matrix and retuned baselines before evaluation.

The defensible outcome is therefore mixed: the newer neural pipeline improves its older neural control on a matched study; readable search is competitive; stronger tuned controls remain difficult; and neither useful inspection nor a gain from every fitted feature is demonstrated.

XV. CONTRIBUTIONS LITERATURE ATTRIBUTION AND LIMITATIONS

We distinguish software reuse, conceptual adaptation, modelling assumptions and original implementation work. PPO, action masking, particle filtering, CEM, residual learning, job dispatch and temporal abstraction are established methods. Calling their combination a new algorithm or claiming world-first status is not supported by the present literature search.

The project-specific contribution is the implemented interaction of costly condition information, job dispatch, deterioration and persistent billing history, together with auditable paired comparisons. We also implement a bounded shared neural scorer, a public service gate, an objective-preserving forced-action wrapper and a selection lock tied to checkpoint identities. These are engineering and empirical contributions in this project, not proof of unique methods worldwide.

The measured neural improvement relative to the older neural control is supported on one matched experimental grid. We do not claim superiority to the stronger tuned rule. Neither better information value, learned service optimisation nor an isolated critic-pretraining benefit has been demonstrated. Negative findings and step-zero selections are central conclusions, not omitted qualifications.

TABLE 10. Source-to-element attribution separates reuse from project-specific work.

Project element	Origin	Relationship
Processing durations	Taillard [1]	Benchmark source
PPO / GAE / masks	[2]–[5], [10]	Existing method / software
Shared scoring	[6]	Conceptual adaptation
CEM / residual	[7], [8]	Established ideas
Temporal abstraction	[9]	Grouped-action motivation
Particle estimates	[11]	Inference concept
Inspection problem	[12]	Related motivation
Critic pretraining	[13]	Controlled adaptation
Hyperheuristics	[14]	Comparison only
Physics / billing / audits	Project [16]	Custom implementation

Existing rules are inexpensive and readable but depend on manually chosen priorities and may overlook long-term interactions. Neural RL can express state-dependent policies but needs representative experience, stable optimisation and strong evaluation. CEM is interpretable and efficient for a small parameter vector, but it searches only that policy family and can converge to weak regions.

Our routes, condition dynamics and tariffs are synthetic. Taillard-derived durations do not supply real deterioration or factory energy measurements. The compressed policy state is approximate; the new 92-value wrapper omits some other-machine and demand-window detail. Candidate shortlists also restrict visible job choices. No exact Markov sufficiency or optimality guarantee follows.

The newer neural studies fix service choices and suppress inspection/discretionary waiting, narrowing the original control problem. Small training-seed counts and validation pools leave substantial selection and training variability. The final fixed-policy test cannot establish population-wide algorithm superiority. Many experiments are exploratory and their intervals are not a family-wise multiple-testing guarantee.

What makes the submission defensible is the explicit information boundary, state/billing persistence, full cost reconstruction, meaningful controls, separate data splits and model selection before final testing. The work is a completed academic research implementation with limits, rather than an industrially validated optimal factory controller.

XVI. CONCLUSION FUTURE WORK AND REPRODUCIBILITY

We built and evaluated a reproducible scheduling research system with uncertain machine condition and electricity demand history. The original adaptive inspection controller did not beat strong rules. Shared candidate scoring produced a functioning PPO dispatch learner that improved over our earlier neural control, while a stronger tuned rule and readable CEM policies remained competitive or better.

Grouping forced actions preserved the physical objective but did not establish a cost gain. Locked validation selection produced a concrete usable neural checkpoint and an independently checked final test. The overall validation winner remains a retuned scheduling rule, even though the selected CEM policy had the lowest final test mean. The report preserves those distinctions and all earlier negative findings.

The practical outcome is a complete, inspectable academic project: implemented simulator, trained models, explicit formulation, source-backed method attribution, raw data, audits and a clear boundary around unestablished claims. Cost is reported as normalised objective performance, never as AI accuracy or industrial money savings.

Long finite horizons retain terminal effects. Particle intervals can under-cover, and correct simulator accounting does not validate physical assumptions. Strong tuned comparators remain difficult to beat. Full GPU execution is slower for this small policy, and the selected mixed implementation also trails pure CPU in the short benchmark.

Future research should prioritise calibrated factory data, richer public metering/machine context, broader workload robustness and separately predeclared learned-service experiments. More training alone is not a guaranteed improvement. These are future directions, not features claimed as completed in this submission.

A. REPRODUCTION AND EVIDENCE

The consolidated entry point is `outputs/inspection_research/report.md`. Primary evidence is stored in full_cycle, residual_dispatch, event_dispatch and final_selection. Each named stage has run.json, episode CSVs, a report and verification evidence. Model folders contain checkpoints, metadata and validation histories. Independent audits are verification_review.json.

The selected neural checkpoint is `outputs/inspection_research/residual_dispatch/models/no_prior_s0/model.zip`. The final selection manifest identifies that exact file and checksum; the selected CEM weights are `priority_search/policy_s1.json`. Overall validation-winning rule parameters are preserved in `final_selection/selection.json`.

To replay completed selection without retraining, run the existing `final_selection/run_selection.py` with the .venv-gpu Python. It verifies identities, reuses completed validation/test caches and regenerates the actual selected-model example. Changed sources or settings require a new named protocol rather than silent cache reuse.

The example JSON contains real candidate jobs, action probabilities and prior-plus-neural logits from a training-split illustrative scenario. It is a decomposition of actual actor output, not a causal feature attribution or an optimality proof. Reviewers should consult the raw episode tables for numerical claims.

B. OBSERVATION DICTIONARY

Full inspection observation: 17 global values; ten jobs each with seven scalar fields and three product indicators; ten machine slots each with 16 scalar fields and three product indicators. Total: 17 + 10(7+3) + 10(16+3) = 307. The action mask is a separate object.

Global fields: elapsed fraction, clock sine/cosine, tariff, billing peak, demand-window average and fraction, waiting and late queue counts, shift remaining, episode remaining, billing-day fraction, periods remaining, selected machine, inspection enabled, direct inspection cost and sensor noise.

Job fields: presence, duration, slack, setup, operations remaining, nominal power and age, followed by product indicators. Machine fields: presence, selected flag, mean/std/5th/95th condition summaries, busy status, remaining work, eligible count, power, continuous-run time, maintenance age, measurement age, measured flag, defer time and PM cooldown, followed by product indicators.

New wrapper: each candidate has duration/60, setup/30, slack/shift clipped to [-4,4], age/1440 clipped to [0,3], operations/max_operations, nominal energy, prior logit and presence. Context contains shift time, episode remaining, tariff, peak, bounded queue and late pressure, selected-machine mean/std/q05/q95, busy fraction and mean remaining work. Total: 10x8 + 12 = 92. Shared actor uses candidate scalars plus context; the no-prior arm ignores prior logits.

Legacy observation: 13 global/selected-machine values plus ten job blocks of ten and ten machine blocks of twelve gives 233. These values belong to the older 11-action configuration and should not be cited as the new policy input size.

XVII. WORKED EXAMPLE AND REFERENCES

C. A worked arithmetic example. The following values are deliberately hypothetical and demonstrate the calculation; they are not experimental results. Suppose one transition adds 24 overdue-job minutes, 40 units of configured energy expense, 300 units of demand liability, 30 service minutes and 100 units of direct inspection expense. The corresponding normalised increments are 0.05, 0.04, 0.10, 0.10 and 0.10. Their sum is 0.39. With no terminal addition, the realised reward is negative 0.39.

If the final transition also leaves two released jobs unfinished, it adds four objective units and makes that transition reward negative 4.39. The example treats the component increments as given for arithmetic; it is not a claim that one inspection alone takes thirty minutes. Actual inspection takes five minutes by default. A real transition may include other elapsed physical activity, and the recorded ledgers determine which service and production minutes actually occurred.

B. An actual saved prediction. The final selected neural checkpoint was replayed on illustrative training scenario 997. At minute 30, machine 0 had two candidate jobs. Job 3 required five processing minutes and eight setup minutes, with 25 minutes of slack. Job 0 required 16 processing minutes, no setup and 60 minutes of slack. Their masked probabilities were approximately 0.497581 and 0.502419. Deterministic evaluation selected job 0, the slightly higher-scoring candidate. This example is an observed model output, not a held-out performance estimate.

The two probabilities are close. They cannot support a claim that the model strongly prefers one job, understands a particular causal tradeoff, or has identified the optimal schedule. The saved example exposes candidate features, final logits, model identity and probabilities. Its no-prior actor has zero prior logits. A longer processing duration can receive a higher neural score because the actor uses several inputs, but the output decomposition alone does not establish which feature caused this particular preference.

The worked reward values are hypothetical arithmetic, while the two-job prediction is an actual saved training-split model output. Neither is a held-out improvement claim. The template and guide supplied by the user control report format and evaluation coverage. Placeholder publication metadata, membership claims, external authors and example figures are not project evidence.

REFERENCES

[1] E. Taillard, Benchmarks for basic scheduling problems. European Journal of Operational Research 64, 278–285; generator and seeds distributed by OR-Library. 1993. [Online]. Available: https://people.brunel.ac.uk/~mastjjb/jeb/orlib/jobshopinfo.html
[2] J. Schulman et al., Proximal Policy Optimization Algorithms. arXiv:1707.06347. 2017. [Online]. Available: https://arxiv.org/abs/1707.06347
[3] S. Huang and S. Ontanon, A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. arXiv:2006.14171. 2020. [Online]. Available: https://arxiv.org/abs/2006.14171
[4] A. Raffin et al., Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR 22(268), 1–8. 2021. [Online]. Available: https://jmlr.org/papers/v22/20-1364.html
[5] Stable-Baselines3 Contrib, Maskable PPO documentation. Software implementation used by this project. 2026 access. [Online]. Available: https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html
[6] C. Zhang et al., Learning to Dispatch for Job Shop Scheduling via Deep Reinforcement Learning. arXiv:2010.12367. 2020. [Online]. Available: https://arxiv.org/abs/2010.12367
[7] I. Szita and A. Lorincz, Learning Tetris Using the Noisy Cross-Entropy Method. Neural Computation 18(12), 2936–2941. 2006. [Online]. Available: https://doi.org/10.1162/neco.2006.18.12.2936
[8] T. Johannink et al., Residual Reinforcement Learning for Robot Control. arXiv:1812.03201. 2018 preprint. [Online]. Available: https://arxiv.org/abs/1812.03201
[9] R. S. Sutton, D. Precup and S. Singh, Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. Artificial Intelligence 112, 181–211. 1999. [Online]. Available: https://doi.org/10.1016/S0004-3702(99)00052-1
[10] J. Schulman et al., High-Dimensional Continuous Control Using Generalized Advantage Estimation. arXiv:1506.02438. 2015 preprint. [Online]. Available: https://arxiv.org/abs/1506.02438
[11] N. J. Gordon, D. J. Salmond and A. F. M. Smith, Novel approach to nonlinear/non-Gaussian Bayesian state estimation. IEE Proceedings F 140(2), 107–113. 1993. [Online]. Available: https://doi.org/10.1049/ip-f-2.1993.0015
[12] D. Koutas, E. Bismut and D. Straub, An investigation of belief-free DRL and MCTS for inspection and maintenance planning. Journal of Infrastructure Preservation and Resilience 5, article 6. 2024. [Online]. Available: https://doi.org/10.1186/s43065-024-00098-9
[13] A. Kernbach et al., Actor-Critic Pretraining for Proximal Policy Optimization. arXiv:2602.23804. 2026. [Online]. Available: https://arxiv.org/abs/2602.23804
[14] S. Lassoued et al., Policy-Based Deep Reinforcement Learning Hyperheuristics for Job-Shop Scheduling Problems. arXiv:2601.11189. 2026. [Online]. Available: https://arxiv.org/abs/2601.11189
[15] PyTorch contributors, Start Locally: official CUDA installation and verification guidance. 2026 access. [Online]. Available: https://pytorch.org/get-started/locally/
[16] Project implementation and experiment artifacts, Factory RL source, run identities, raw episode tables, checkpoint metadata and independent reviews. Local primary evidence, frozen 2 November 2026. 2026. Local: `outputs/inspection_research/` and factory_rl/
[17] User-supplied course handout, “Performance metric and report format guidance, extitPPTperfromance.pdf, 20 pages, n.d. Evaluation guidance and examples, not experimental evidence.

XVIII. APPENDIX A FULL INSPECTION POMDP DESIGN

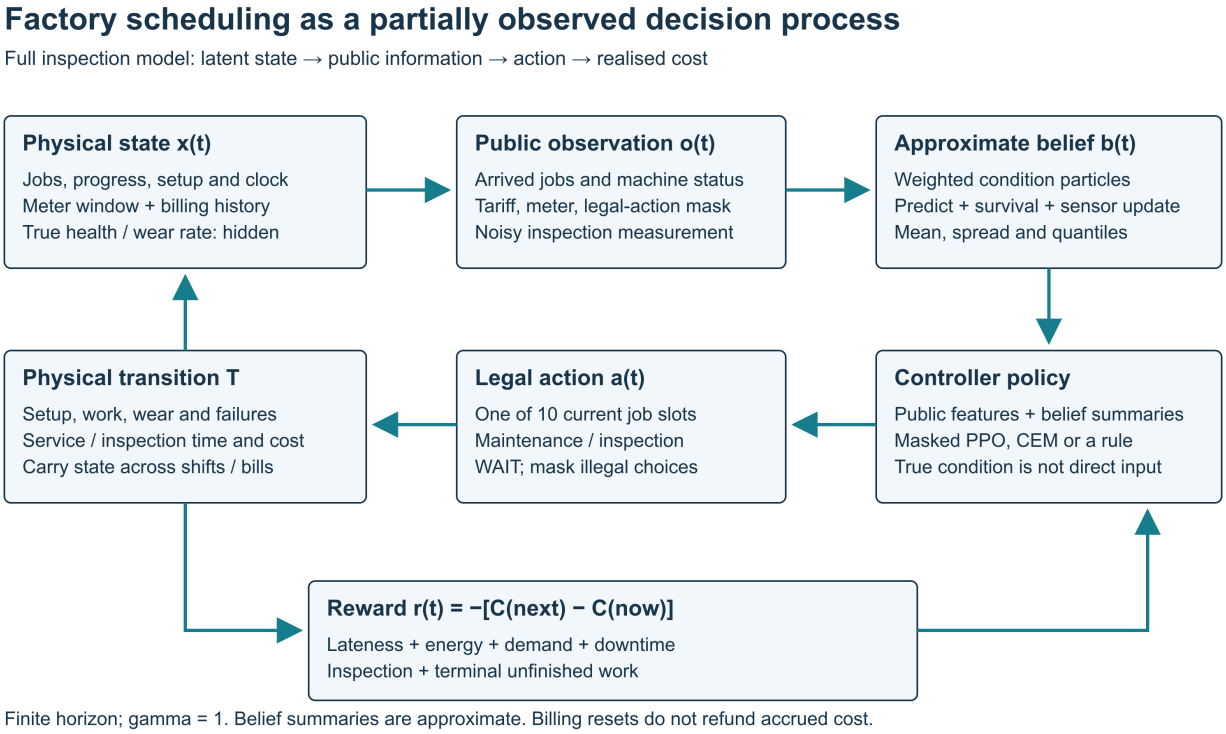


FIGURE 4. Implemented information and action loop. Public observations and approximate particle beliefs are separated from latent physical condition.

A. Why the condition is hidden. An ordinary scheduler can observe machine status and work history without knowing the exact latent deterioration state. In the full inspection formulation, true health and wear rate drive physical outcomes but are not sent directly to the ordinary controller. Inspection yields a noisy condition measurement rather than perfect truth. The problem is therefore partially observed. A privileged exact-health experimental condition is useful as a diagnostic comparison, but its policy is not an optimal bound. [11], [12], [16]

B. What the controller receives. The public policy interface exposes summaries such as estimated mean, standard deviation and lower and upper condition quantiles, together with measurement and service history. The underlying particle population is richer than these summaries. Two different distributions can share similar summaries yet imply different failure risks. Compressing the belief into a small vector consequently trades representational detail for simpler learning and cannot be taken as proof that the observation is an exact Markov state.

C. What execution changes. After a legal dispatch, the machine performs any required product setup and advances the selected operation. Physical integration accumulates electricity use, wear, failure exposure, elapsed time and overdue-job minutes. Other machines may continue working during the interval. If an operation finishes, precedence may make the next operation eligible. If the machine fails, corrective repair consumes time and interrupted work resumes after repair. The next decision therefore depends on the realised transition, not only the original duration estimate.

D. Why incremental reward is useful. The reward is the negative increase in cumulative objective between consecutive decisions. With finite-horizon discount one, summing rewards cancels intermediate cumulative costs and gives negative total final cost, assuming the initial cumulative cost is zero. A maintenance or inspection action receives no special bonus. Its value must come from improved subsequent physical or scheduling outcomes after paying its own time and expense.

XIX. APPENDIX B CONSTRAINED NEURAL CONTROLLER

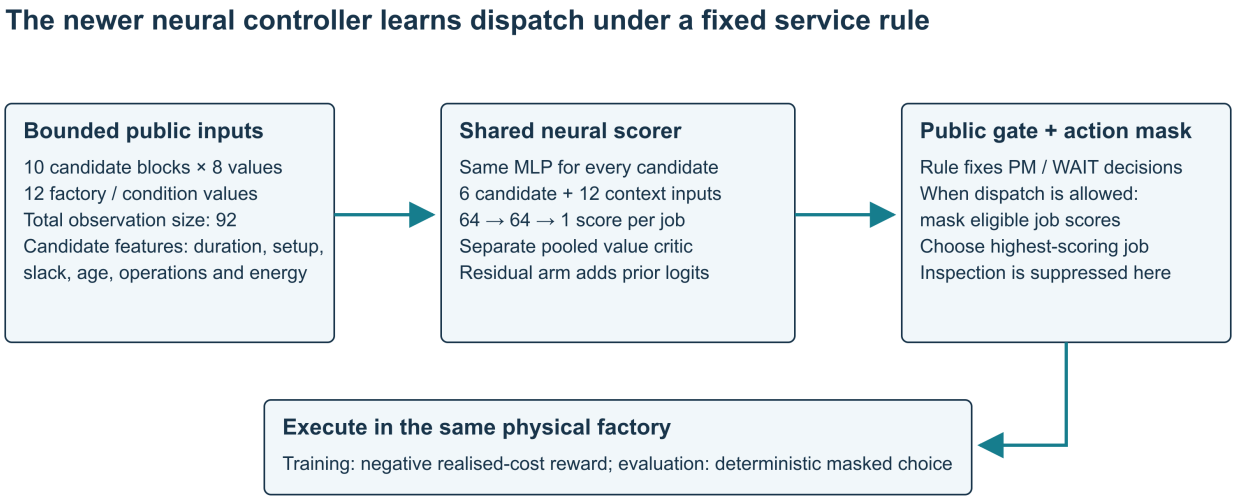


FIGURE 5. The newer shared actor learns job dispatch while a public rule controls maintenance and waiting. Inspection is suppressed in this wrapper.

C. Why share a scorer. The newer actor evaluates every candidate with the same small network. Six candidate scalars and twelve context values enter two hidden layers of sixty-four units, followed by one job score. A separate pooled critic estimates value. Weight sharing lets a feature such as setup time have the same learned interpretation across shortlist positions. Bounded inputs reduce extreme scales, but neither boundedness nor sharing proves that the compressed observation contains all information needed for optimal control.

The no-prior arm learns job scores without initial priority logits. The residual arm adds a bounded neural correction to a previously validation-retuned setup-aware SPT prior. Its correction head starts at zero, so the initial deterministic residual policy exactly reproduces the rule. Stochastic training can still explore other choices. A bounded correction may reverse close rankings while preventing departures across large prior-score gaps; this restriction can stabilise behaviour but also limit improvement. [6], [8], [16]

D. Checkpoint selection is part of the method. Validation evaluates saved checkpoints, including step zero. In the original study step zero means the imitation warm start, not a learned PPO improvement. In the residual dispatch study it means the initial rule-assisted scorer. All three residual seeds retained step zero. The no-prior seeds selected steps 65536, 57344 and 16384, demonstrating that their selected policies actually included PPO training. Their achieved costs still require comparison with strong controls.

E. Grouped actions and hardware. Grouping consecutive decisions with only one allowed action preserves realised actions and summed rewards, but changes the GAE time index and update frequency. The grouped experiment therefore matches a physical-action target and records rollout overshoot. Training uses CPU simulation and small inference calls with CUDA batch updates. Actual GPU use was verified; the short throughput benchmark favoured pure CPU, so GPU execution is not presented as a measured speedup or a cause of better policy quality.

E. Priorities for further work. The strongest next scientific step is to improve external validity and identify a mechanism through a predeclared experiment. Useful directions are measured factory calibration, richer public metering and other-machine context, broader workload transfer, and separately optimised learned-service policies. Future controller changes should preserve this evidence and use new named studies with strong retuned comparators. More training alone cannot guarantee improvement. For a project review, the defensible claim is that the system is complete and inspected, the newer neural baseline improves its predecessor, and strong rules and readable search remain difficult to beat.